

Multi-UAVs Network Design Algorithms for Computed Rate Maximization

Kefeng Wu^{ID}, Kwan-Wu Chin^{ID}, and Sieteng Soh^{ID}, *Member, IEEE*

Abstract—This paper considers a network design problem using Unmanned Aerial Vehicles (UAVs). It aims to create a network to provide communication and computation service to a set of source-destination ground node pairs. The main performance metric is the minimum amount of computed data among a set of source-destination pairs. To optimize this metric, we outline two mixed Integer Linear Programs (MILPs), namely S-MILP and NS-MILP, which are designed respectively for splittable and non-splittable traffic flow models. They jointly optimize the placement of UAVs, assignment of Virtualized Network Functions (VNFs), and routing of unprocessed and processed flow. Further, NS-MILP optimizes the path selection of each source-destination pair. A key challenge is that these MILPs require an exhaustive collection of network topologies. To this end, this paper outlines two heuristic algorithms, called Resource-Aware Location Selection (RALS) and Resource-Aware Path and Location Selection (RAPLS), respectively for each traffic flow model. The simulation results show that RALS and RAPLS achieve on average 83% and 80% of the amount of computed flow of S-MILP and NS-MILP, respectively. Lastly, RALS and RAPLS require 45% and 53% less computation time as compared to S-MILP and NS-MILP, respectively.

Index Terms—Drones, forwarding, multi-commodity flow, virtual machines, virtualization.

I. INTRODUCTION

NETWORK Function Virtualization (NFV) is a technology that addresses the high operating cost and growing complexity in managing network infrastructures [1]. Specifically, current networks have specialized network equipment, aka middleboxes, that run functions such as firewalls, Network Address Translators (NATs) and proxies. As noted in [2], these middleboxes require dedicated hardware and high expenses to deploy and maintain. In addition, they are not readily extensible. In contrast, NFV allows an operator to implement and deploy software version of these middleboxes as Virtualized Network Functions (VNFs) that run on one or more off-the-shelf servers by utilizing Virtual Machines (VMs) [3] or Docker containers [4].

Manuscript received 5 September 2023; revised 17 December 2023; accepted 15 January 2024. Date of publication 18 January 2024; date of current version 6 August 2024. Recommended for acceptance by M. Segata. (Corresponding author: Kefeng Wu.)

Kefeng Wu and Kwan-Wu Chin are with the School of Electrical, Computer and Telecommunications Engineering, University of Wollongong, Wollongong, NSW 2522, Australia (e-mail: kw205@uowmail.edu.au; kwanwu@uow.edu.au).

Sieteng Soh is with the School of Electrical Engineering, Computing, and Mathematical Sciences, Curtin University, Perth, WA 6102, Australia (e-mail: s.soh@curtin.edu.au).

Digital Object Identifier 10.1109/TMC.2024.3355391

Another network equipment that is becoming popular is Unmanned Aerial Vehicles (UAVs) [5]. In particular, UAVs have been used to extend the coverage of a network [6], where they act as relays or mobile base stations, and provide in-situ computation services to ground nodes [7]. Further, UAVs can host VNFs, where a UAV may provide Domain Name System (DNS) services to ground nodes [8]. Alternatively, UAVs can run a firewall or intrusion detection system to protect and enhance the security of devices [9].

Given the aforementioned technologies, a fundamental question arises: for a given fixed set of source nodes, how much of their traffic can be routed and processed by VNFs? In this paper, we answer this question for a multi-UAVs network. Specifically, our aim is to construct a multi-UAVs network that maximizes the minimum computed flow rate among a given set of ground node pairs. This question is significant. First, it is an open research question. Second, its answer informs a network operator the network configuration/topology that yields the highest minimum amount of processed traffic given source-destination pairs, and also how to utilize the computation and communication resources of UAVs.

To illustrate our research question, consider Fig. 1(a) or Fig. 1(b). We see a backbone network constructed using two UAVs that provide connectivity to two source-destination node pairs (A, B) and (C, D) with a fixed location. The traffic from each source must be processed by one or more VNFs before arriving at its destination. Consequently, there are two types of traffic over each link: i) *unprocessed*, and ii) *processed*. A traffic becomes processed after it is routed through a VNF. For example, in Fig. 1(a), we see that source-A's traffic is processed by VNFs on UAV-1 before arriving at destination-B, and source-C's traffic is processed by UAV-2 before arriving at destination-D.

The key performance metric of interest is the minimum processed/computed flow rate among all source-destination nodes, which is the minimum amount of computed flow that can be sent across all source-destination nodes. We consider this metric to ensure the flow rate of each source node is strictly positive. To maximize this metric, we must construct a topology, determine the routing of traffic, and allocate computation and communication resources. Referring to Fig. 1(a), communication resources are required to support the traffic from each source. Further, both UAVs must allocate computation resources if they run a VNF. Further, we need to consider the computed flow rate of different topologies. Specifically, each topology yields different routes between source-destination nodes, which in turn increase/decrease the available communication resources

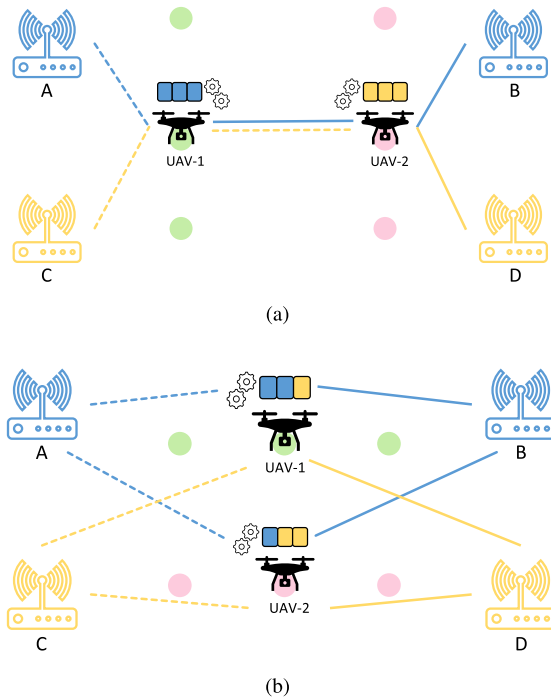


Fig. 1. Two possible topologies with different link constructions and VNFs assignments. Each UAV has a set of *potential* locations, denoted as green and pink circles. Further, each UAV hosts three VNFs that are denoted as boxes. Note that an assigned VNF has the color blue or yellow corresponding to a source-destination node pair. In addition, we use dashed lines to denote unprocessed flows and solid lines to denote processed flows. (a) A possible topology with a shared link. (b) A possible topology without shared links.

to these nodes. Hence, a key goal is to identify the topology that has the highest minimum computed flow rate, aka the max-min computed flow rate or the max-min computed flow for short.

To illustrate the joint topology construction, routing, and resource allocation problem, consider Fig. 1. For ease of exposition, we assume all links have a capacity of 5 Mbits/s and each VNF can process one unit of data per second. In Fig. 1(a), the max-min computed flow rate among two source-destination node pairs is 2.5 Mbits/s because both sources share the same UAV-to-UAV link. On the other hand, in Fig. 1(b), the max-min compute flow rate is 3 Mbits/s. This is because each source-destination node is using a separate communication path; i.e., each link is dedicated to a source-destination node pair. In this example, however, the amount of *processed* traffic is not limited by communication resources but computation. Observe that each source-destination node is assigned three VNFs, which equates to three units of processed flow. Although each link has a capacity of 5 Mbits/s, the max-min computed flow rate is 3 Mbits/s.

According to the previous example, there are three quantities to optimize: i) the placement of UAVs, ii) the assignment of VNFs, and iii) the routing of processed/unprocessed flow. The problem at hand is challenging as the number of topologies grows exponentially with the number of UAVs and *potential* locations. For example, if there are Z *potential* locations and N

UAVs, then there are a total of $\binom{Z}{N}$ possible topology configurations. In addition, a sub-optimal assignment of VNFs and routing paths may result in bottleneck links and unused VNFs. Another critical issue is whether traffic can be split across multiple paths, namely *splittable* or *non-splittable* traffic. In the *non-splittable* flow case, each source-destination node pair is only allowed to use one path. Therefore, determining the path of each source-destination node pair is another challenge. This is because for an arbitrary topology, the total number of paths is proportional to the number of UAVs and the number of source-destination node pairs.

Henceforth, this paper makes the following contributions:

- We outline a novel joint UAVs placement, VNFs assignment and routing problem that aims to construct a multi-UAVs backbone network such that the minimum amount of computed data from a set of source nodes to their intended destinations is maximized. We consider both *splittable* and *non-splittable* flow case. We then formulate a Mixed Integer Linear Program (MILP) for both flow models. These MILPs yield the optimal location of UAVs, the optimal VNFs assignment as well as the optimal routing path of *unprocessed/processed* traffic flow. Advantageously, they can be used to compute the theoretical computed flow rate upper bound, which then serves as a benchmark for use by future solutions.
- We outline heuristic solutions for both flow models. First, for the *splittable* flow case, we outline Resource-Aware Location Selection (RALS). Then, for the *non-splittable* flow case, we outline Resource-Aware Path and Location Selection (RAPLS). These heuristic solutions do not require an exhaustive collection of topologies.
- We present the first study of the problem at hand and two heuristic solutions. Our study compares the amount of computed data obtained by RALS and RAPLS against their corresponding MILP with different parameters, such as the number of UAVs, number of source-destination node pairs and number of VNFs per UAV. We also study RALS and RAPLS in large networks.

Next, Section II discusses related works. Then Section III presents the system model followed by the problem formulation in Section IV. Section V introduces two heuristic solutions. Simulation results are presented in Section VI. Lastly, Section VII concludes the paper.

II. RELATED WORKS

Our work overlaps with the following research areas: i) middleboxes/VNFs placement and routing with joint communication and computation consideration, and ii) deploying VNFs in multi-UAVs networks. Both areas consider communication and computation resource allocation. They, however, do not study the same system or research aim, i.e., they either assume a fixed network topology or they do not aim to maximize the minimum computed flow rate of a network. Next, we discuss each of these research areas separately.

There are many works that consider VNFs placement [10], [11], [12], routing [13], [14], [15], and joint VNFs

placement and routing [16], [17], [18]. Specifically, the work in [10]–[12] studies the placement of VNFs but target different criteria. In [10], VNFs are placed at the edge of a network near end users so as to minimize end-to-end delay. The work in [11] considers selecting a subset of nodes to host VNFs in order to maximize the total amount of processed traffic. The cost of hosting a VNF and the limited processing capacity of a node are jointly considered. In [12], VNFs availability and reliability are evaluated under different VNFs placement strategies, e.g., one host node runs all VNFs, each host node runs one VNF or some nodes run one or more VNFs while other nodes just function as a relay. Note that these works assume that traffic flows on a predetermined path. Thus, their concern is only on VNFs placement to achieve some objective.

Routing is a critical issue after VNFs have been deployed at desired locations. For example, works such as [13] and [14] study routing with unprocessed/processed flows subject to links capacity and nodes processing capacity. Both works aim to maximize the amount of processed flow in a network. However, reference [13] considers traffic volume changes, and the work in [14] assumes incoming traffic and outgoing traffic are the same for a processing node. Note that the work in [14] is close to our work in terms of routing. This is because it assumes the traffic from a source node can be processed at any node with available processing capacity before arriving at its intended destination. However, it does not involve VNFs placement or network construction. In a different work, in [15], the authors aim to design efficient routing algorithms to route and process a set of traffic flows. The main contribution of [15] is that both splittable and non-splittable flows are taken into consideration when designing a routing algorithm.

There are works that jointly consider VNFs placement and routing [16]–[18]. These works solve the placement and routing problem on top of a substrate network, meaning the network topology is given. For example, in [16], traffic volume changes when a VNF processes incoming traffic. They aim to minimize the maximum link load by determining VNFs placement and routing in a tree topology. Moreover, the work in [17] not only considers load balancing but also considers the deployment cost of VNFs and links. It aims to ensure multicast traffic from a source node to multiple destination nodes are processed by VNFs with minimum cost, i.e., the total number of deployed VNFs and the total number of physical links. Another work in [18] considers additional requirements such as VNF complexity and maximum tolerable end-to-end delay of users. The VNF complexity is defined as the task completion time of different types of VNFs with respect to different services. The aim of this work is to maximize the number of users who received service and to minimize the total cost relating to VNF setup time, VNF complexity, and end-to-end delay of users.

Deploying VNFs on UAVs has a number of benefits, see [19] for a survey. Works within this topic can be divided into two categories: i) fixed network topology [20], [21], [22], and ii) unknown network topology [23]. For the first category, UAVs are connected over a single-hop or multi-hop connection. With this assumption, the work in [20] considers minimizing the energy consumption of a FANET that provides services and functions

TABLE I
COMPARISON OF RELEVANT WORKS

Reference	Multi-UAVs	Network construction	Routing	Non-splittable flow	VNFs placement/assignment	Optimize computed flow rate
[11]	×	×	×	×	×	✓
[10], [12]	×	×	×	×	✓	×
[13], [14]	×	×	✓	×	×	✓
[15]	×	×	✓	✓	✓	×
[16], [17], [18]	×	×	×	×	✓	×
[20], [21]	✓	×	×	×	✓	✓
[22]	✓	×	×	×	×	✓
[23]	✓	✓	✓	×	✓	×
Our work	✓	✓	✓	✓	✓	✓

to ground users. The placement of VNFs and service function chaining is determined based on the service requirement of users, the processing capacity of UAVs, and the available onboard energy of UAVs. Another work in [21] considers a two-tier UAVs network that serves ground users. There is a master UAV that has all VNFs. These VNFs are ready to be instantiated on a fleet of slave UAVs to serve ground users. A user may request different services represented as a service function chain. In this regard, the solution in [21] establishes a path for the said chain, and determine a schedule that minimizes the completion time of all tasks from ground users. In [22], a team of UAVs forms a linear topology to serve one end-to-end request. Specifically, there is only one request generated by one source node to its paired destination node. VNFs placement and UAVs trajectory are jointly considered to minimize the total energy consumption of UAVs as well as the end-to-end delay. The main difference between the aforementioned works and our work is that these works do not consider a network construction problem. These works can be seen as an extension of VNFs placement problem in wired/wireless networks with an additional energy consumption constraint brought by UAVs.

In terms of UAV network construction problem, the work in [23] aims to determine the location of UAVs to create physical links. In addition, VNFs placement and service function chaining are also determined by considering the required data rate of user requests. However, the authors of [23] aim to minimize the number of deployed UAVs such that the user requests are satisfied. In contrast, we do not aim to minimize the number of deployed UAVs and our research aim is to maximize the minimum amount of processed flow among a set of source-destination pairs.

Table I summarizes discussed works. We see that only [23] has considered network construction. Reference [15] considers routing of splittable and non-splittable flows over an existing network with various computational services. To the best of our knowledge, our study is the first that jointly considers network construction, VNFs assignment, and unprocessed/processed flow routing in a multi-UAVs network. In addition, we optimize the minimum computed flow rate of the network under both splittable and non-splittable flow models.

III. SYSTEM MODEL

A. Network Model

Table II summarizes key notations. There are K source-destination pairs supported by U rotary-wing UAVs. Define set $\mathcal{S} = \{s_k \mid k = 1, 2, \dots, K\}$ to contain fixed source nodes,

TABLE II
KEY NOTATIONS

1.	Sets
$\mathcal{U}$	The set of UAVs
$\mathcal{S}$	The set of source nodes
$\mathcal{D}$	The set of destination nodes
$\mathcal{K}$	The set of source-destination pairs
$\mathcal{V}$	The set of nodes
$\mathcal{E}_i$	The set of p-links in graph G_i
$\mathcal{Z}$	The set of <i>potential</i> locations
$\mathcal{Q}_u$	The set of VMs at UAV u
2.	Constants
P_{max}	The maximum transmit power
h	The height of UAVs
W	System bandwidth
γ	Path loss exponent
η_{LoS}	The excessive attenuation factor of LoS condition
η_{nLoS}	The excessive attenuation factor of NLoS condition
β	SNR threshold
N_0	Ambient noise power (Watt/Hz)
3.	Variables
A	Size of deployment area
ψ	Number of grid lines on each side of deployment area
R	Transmission range
P_v	Transmit power of all nodes
P_{ju}	Received power at UAV u for link (j, u)
$\tilde{L}(s_k, z_u)$	Path loss of ground-to-UAV p-links
$\tilde{L}(z_j, z_u)$	Path loss of ground-to-UAV p-links
$d(s_k, z_u)$	Euclidean distance between node s_k and u
$B(e_i)$	Link capacity of link e_i
$C(u)$	Processing capacity of UAV u
$f_k(e_i)$	The amount of flow on link e_i that belongs to the k -th source-destination pair
$w_k(e_i)$	The amount of unprocessed flow on link e_i that belongs to the k -th source-destination pair
$p_k(v)$	The amount of flow processed at node v that belongs to the k -th source-destination pair

and the set $\mathcal{D} = \{d_k \mid k = 1, 2, \dots, K\}$ contains fixed destination nodes. Let $\mathcal{K} = \{k \mid k = 1, 2, \dots, K\}$ denote the set of source-destination pairs. Each source-destination pair $k \in \mathcal{K}$ has a maximum demand of M_k . Further, each source node s_k and destination node d_k have a height of zero with a fixed location. Their coordinate is respectively $(x_{s_k}, y_{s_k}, 0)$ and $(x_{d_k}, y_{d_k}, 0)$.

Define $\mathcal{U} = \{u \mid u = 1, 2, \dots, U\}$ as the set of UAVs. They operate on a $A \times A$ m^2 square area at height h . The area is divided into a grid. There are a total of $\psi \times \psi$ grid lines on the area and each intersection of grid lines is a *potential* location to place a UAV. Let $\mathcal{Z} = \{z \mid z = 1, 2, \dots, Z\}$ denote the set of *potential* locations, where there are a total of $|\mathcal{Z}| = \psi \times \psi$ *potential* locations. Let (x_u, y_u, h) denote the 3D Cartesian coordinate of UAV u .

There are two types of links, namely *potential* links (p-links) and *active* links. A p-link is defined as a link whereby an end-point does not have a UAV. Specifically, a p-link exists between a source node in $\mathcal{S}$ and a *potential* location in $\mathcal{Z}$, or between a *potential* location in $\mathcal{Z}$ and a destination node in $\mathcal{D}$, or between two *potential* locations in $\mathcal{Z}$. For example, in Fig. 2, p-links are dashed black lines. Accordingly, there are two types of p-links: ground-to-UAV p-links and UAV-to-UAV p-links. For example, consider source node s_k , destination node d_k , two *potential* locations z_j and z_u , denote a ground-to-UAV p-link as (s_k, z_j) or (z_u, d_k) , and denote a UAV-to-UAV p-link as (z_j, z_u) . When a UAV has been placed at a *potential* location

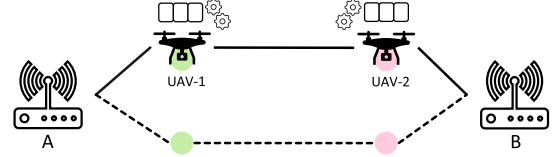


Fig. 2. Example of *active* links and p-links. Node A and B are respectively a source node and a destination node. Green and pink circles are *potential* locations. Two UAVs have selected their locations. Thus, black solid lines are *active* links and black dotted lines are p-links.

in $\mathcal{Z}$, all ground-to-UAV p-links entering/leaving this location become *active* links. Similarly, when two UAVs have selected their location in $\mathcal{Z}$, the UAV-to-UAV p-link between these two locations becomes an *active* link.

Let $\mathcal{G} = \{G_i(\mathcal{V}, \mathcal{E}_i) \mid i = 1, 2, \dots, |\mathcal{G}|\}$ denote the collection of topologies, where each topology is denoted as $G_i(\mathcal{V}, \mathcal{E}_i)$. For each topology $G_i(\mathcal{V}, \mathcal{E}_i)$, $\mathcal{V}$ is the set of nodes and $\mathcal{E}_i$ is the set of p-links. The set of nodes is $\mathcal{V} = \mathcal{S} \cup \mathcal{D} \cup \mathcal{U}$. Specifically, for graph G_i , let $v \in \mathcal{V}$ denote a node, where v can be a source node, a destination node or a UAV with a selected location. Let $e_i \in \mathcal{E}_i$ denote a p-link. Moreover, define $\delta_i^+(v)$ as the set of p-links leaving node v . Define $\delta_i^-(v)$ as the set of p-links entering node v . Note that, the total number of topologies $|\mathcal{G}|$ depends on the number of UAVs and the potential locations. For example, if there are U UAVs and a total of Z potential locations, then we have a total of $\binom{Z}{U}$ possible topologies.

We aim to select only one topology that maximizes the minimum amount of processed flow among all source-destination pairs. Let $\alpha_i \in \{0, 1\}$ denote whether topology G_i is selected. As only one topology can be selected, we have

$$\sum_{i=1}^{|\mathcal{G}|} \alpha_i = 1. \quad (1)$$

B. Link Activation Model

To activate topology G_i , we need to determine the placement of UAVs which in turn determines the set of *active* links. Define $a_{u,i}^z \in \{0, 1\}$ to indicate whether *potential* location z is selected by UAV u in G_i . Each UAV selects a location in $\mathcal{Z}$. Formally, we have

$$\sum_{n=1}^N a_{u,i}^z = \alpha_i, \quad \forall G_i \in \mathcal{G}, \quad \forall u \in \mathcal{U}. \quad (2)$$

Let $I(e_i) \in \{0, 1\}$ indicate whether link e_i is active. For a ground-to-UAV p-link (s_k, z) , we have

$$I(s_k, z) \leq a_{u,i}^z, \quad \forall G_i \in \mathcal{G}, \quad \forall u \in \mathcal{U}, \quad (3)$$

meaning p-link (s_k, z) becomes *active* only when UAV u selects *potential* location z .

For a set of UAV-to-UAV p-links $\{(z_j, z_u) \mid z_j, z_u \in \mathcal{Z}\}$ that involve UAV j and u , the activation of link (z_j, z_u) is constrained by

$$I(z_j, z_u) \leq a_{j,i}^z a_{u,i}^z, \quad \forall G_i \in \mathcal{G}, \quad \forall j \in \mathcal{U}, \quad \forall u \in \mathcal{U}, \quad (4)$$

meaning p-link (z_j, z_u) becomes an *active* link when UAV j and u are placed at location z_j and z_u , respectively.

Note that inequality (4) is nonlinear. In order to linearize it, let $Y_{j,u}^{z,i} = a_{j,i}^z a_{u,i}^z$, where $Y_{j,u}^{z,i} \in \{0, 1\}$. Inequality (4) is then revised to

$$I(z_j, z_u) \leq Y_{j,u}^{z,i}, \forall G_i \in \mathcal{G}, \forall j \in \mathcal{U}, \forall u \in \mathcal{U}. \quad (5)$$

Moreover, the value of $Y_{j,u}^{z,i}$ is restricted by

$$Y_{j,u}^{z,i} \leq a_{j,i}^z, \quad (6a)$$

$$Y_{j,u}^{z,i} \leq a_{u,i}^z, \quad (6b)$$

$$Y_{j,u}^{z,i} \geq a_{j,i}^z + a_{u,i}^z - 1. \quad (6c)$$

Observe that $Y_{j,u}^{z,i} = 1$ only when $a_{j,i}^z = 1$ and $a_{u,i}^z = 1$.

C. Channel Model

For ground-to-UAV p-links, we use the probabilistic path loss model of [24]; other channel models can be used as long as they produce a deterministic path loss or channel power gain value. Specifically, there are two propagation groups: i) Line-of-Sight (LoS) and ii) Non-LoS (NLoS) conditions. The LoS and NLoS probability of a ground-to-UAV p-link (s_k, z_u) is respectively

$$p^{LoS}(s_k, z_u) = \frac{1}{1 + a \exp[-b(\theta_{ku} - a)]}, \quad (7a)$$

$$p^{nLoS}(s_k, z_u) = 1 - p^{LoS}(s_k, z_u), \quad (7b)$$

where a and b are constants that depend on the environment, and θ_{ku} denotes the elevation angle between source node d_k and UAV u . Formally, it is defined as $\theta_{ku} = \arcsin \frac{h}{d(s_k, z_u)}$, where $d(s_k, z_u)$ is the Euclidean distance between source node s_k and location z_u . The ground-to-UAV path loss consists of two parts: (i) free space path loss, and (ii) additive path loss due to the shadowing and scattering effects [24]. Let $\hat{L}^{LoS}(s_k, z_u)$, and $\hat{L}^{nLoS}(s_k, z_u)$ respectively denote the LoS and NLoS path loss of a ground-to-UAV p-link (s_k, z_u) . We have

$$\hat{L}^{LoS}(s_k, z_u) = 10 \log_{10} \left(\frac{4\pi f_c d(s_k, z_u)}{c} \right)^\gamma + \eta_{LoS}, \quad (8a)$$

$$\hat{L}^{nLoS}(s_k, z_u) = 10 \log_{10} \left(\frac{4\pi f_c d(s_k, z_u)}{c} \right)^\gamma + \eta_{nLoS}, \quad (8b)$$

where f_c is the carrier frequency, c is the speed of light, γ is the path loss exponent, η_{LoS} and η_{nLoS} are excessive attenuation factors of LoS and NLoS condition, respectively.

Let $\hat{L}(s_k, z_u)$ denote the average path loss of a ground-to-UAV p-link (s_k, z_u) , which is

$$\begin{aligned} \hat{L}(s_k, z_u) &= p^{LoS}(s_k, z_u) \times \hat{L}^{LoS}(s_k, z_u) \\ &\quad + p^{nLoS}(s_k, z_u) \times \hat{L}^{nLoS}(s_k, z_u). \end{aligned} \quad (9)$$

We assume UAVs are able to maintain LoS links [25] and there is only free space path loss for UAV-to-UAV links. Let $\tilde{L}(z_j, z_u)$ denote the path loss between the selected location of

UAV j and u , where

$$\tilde{L}(z_j, z_u) = 10 \log_{10} \left(\frac{4\pi f_c d(z_j, z_u)}{c} \right)^\gamma. \quad (10)$$

D. Communication Model

Let W be the bandwidth. All links operate on an orthogonal frequency to avoid ground-to-UAV and UAV-to-UAV interference; a method to assign frequencies is [26]. We also assume UAVs are equipped with full-duplex¹ radios which means they can transmit and receive data at the same time [28]. For an implementation of a full-duplex radio, see [29]. All nodes use transmit power P_v (in dBm). Let P'_{ju} denote the received power at the location of UAV u (in dBm) from node j . We have

$$P'_{ju} = P_v - \tilde{L}(z_j, z_u). \quad (11)$$

For a receiver to successfully decode the message from a transmitter, its Signal-to-Noise Ratio (SNR) must be above a given threshold β . Let P_{ju} denote the received power at the location of UAV u (in Watt) from node j . Formally, we have

$$\frac{P_{ju}}{N_0 W} \geq \beta, \quad (12)$$

where N_0 is the ambient noise power spectrum density.

The theoretical link capacity of link $(z_j, z_u) \in \mathcal{E}_i$ is

$$B(z_j, z_u) = W \log_2 \left(1 + \frac{P_{ju}}{N_0 W} \right). \quad (13)$$

E. Flow Model

For each topology G_i , each link e_i has capacity $B(e_i)$. Define $f_k(e_i)$ as the amount of flow on link e_i that belongs to the k -th source-destination pair. The total flow on a link must not exceed its capacity:

$$0 \leq \sum_{k=1}^K f_k(e_i) \leq B(e_i), \forall e_i \in \mathcal{E}_i. \quad (14)$$

There are two flow routing schemes: i) splittable, and ii) non-splittable. For the splittable flow case, traffic from a source node can be routed over multiple paths. However, for the non-splittable flow case, traffic from a source node is only allowed to use one path. We will present the flow conservation constraint for both flow routing schemes.

1) *Splittable Flow Conservation*: For the k -th source-destination pair, at UAV u , we have

$$\sum_{e_i \in \delta_i^-(u)} f_k(e_i) = \sum_{e_i \in \delta_i^+(u)} f_k(e_i), \forall k \in \mathcal{K}, \forall u \in \mathcal{U}. \quad (15)$$

2) *Non-Splittable Flow Conservation*: Define $\mathcal{P}_k^i = \{p \mid p = 1, 2, \dots, P_k\}$ as the set of paths between source-destination pair k in topology G_i , where P_k is the maximum number of paths for source-destination pair k . Let $S_k^i(p) \in \{0, 1\}$ indicate whether

¹Note that one can also assume UAVs employ a single radio and operate over the same frequency. In this case, methods such as [27] can be applied to jointly optimize routing and link scheduling.

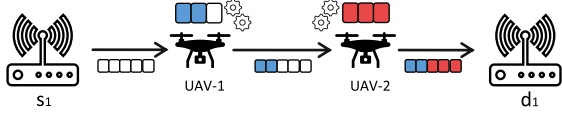


Fig. 3. Example of flow computation involving two UAVs and one source-destination pair. A series of boxes under each arrow denotes the traffic from source node s_1 . Blank boxes mean unprocessed traffic and colored boxes mean processed traffic. Further, each UAV hosts three VNFs for flow computation, denoted as three boxes above a UAV. The flow processed by UAV-1 has the color blue and the flow processed by UAV-2 has the color red.

path p of source-destination pair k in topology G_i is selected. We have

$$\sum_{p=1}^{P_k} S_k^i(p) = 1, \forall G_i \in \mathcal{G}, \forall k \in \mathcal{K}. \quad (16)$$

Constraint (16) ensures each source-destination pair only selects one path in topology G_i . Thus, the flow conservation constraint in the non-splittable flow case is

$$\sum_{p=1}^{P_k} \sum_{e_i \in \delta_i^-(u)} S_k^i(p) f_k(e_i) = \sum_{p=1}^{P_k} \sum_{e_i \in \delta_i^+(u)} S_k^i(p) f_k(e_i), \quad \forall G_i \in \mathcal{G}, \forall k \in \mathcal{K}, \forall u \in \mathcal{U}. \quad (17)$$

Constraint (17) ensures traffic flow is only routed via links that are on the selected path.

F. Computation Model

Traffic flow from source node s_k must be fully processed by one or more UAVs before arriving at destination node d_k . Therefore, it is necessary to quantify the amount of flow processed at each UAV and the amount of unprocessed flow on each link e_i . Referring to Fig. 3, we see that s_1 transmitted five units of unprocessed flow. UAV-1 and UAV-2 processed respectively two and three units of flow, which ensures the traffic is fully processed upon arrival at d_1 .

Next, we will model processed and unprocessed flow as it traverses one or more paths from its source node to its destination node.

Each UAV u has a maximum of Φ_u Virtual Machines (VMs).² Note that the value of Φ_u is determined by the available energy on UAV u . Further, our work does not consider migration of VMs between UAVs as our goal is to instantiate as many VMs on one or more UAVs to process the flow of each source-destination pair. Define $\mathcal{Q}_u = \{\phi \mid \phi = 1, 2, \dots, \Phi_u\}$ as the set of VMs of UAV u . Each VM operates at frequency f_{vm} (Hz). We assume all data that arrives at each UAV has a processing time of one second. As per [30], we assume processing one bit of data requires Γ CPU cycles. Thus, the processing capacity of each VM is $\frac{f_{vm}}{\Gamma}$ (bps). Therefore, for each UAV u , its processing capacity $C(u)$

is

$$C(u) = \sum_{\phi=1}^{\Phi_u} \left(\frac{f_{vm}}{\Gamma} \right), \forall u \in \mathcal{U}. \quad (18)$$

Define $X_\phi^{UK} \in \{0, 1\}$ to indicate whether VM ϕ of UAV u is assigned to source-destination pair k . Each VM on a UAV can be assigned to only one such pair. Formally, we have

$$\sum_{k=1}^K X_\phi^{UK} = 1, \forall \phi \in \mathcal{Q}_u, \forall u \in \mathcal{U}. \quad (19)$$

Note that a source-destination pair k can be served by multiple VMs; i.e., it can be assigned multiple VMs on a UAV.

Define $p_k(u) \geq 0$ as the amount of flow processed at UAV u that belongs to the k -th source-destination pair. Considering the processing capacity of each VM, for each source-destination pair k , the amount of data processed at UAV u cannot exceed the total processing capacity of assigned VMs. Thus, we have

$$p_k(u) \leq \sum_{\phi} X_\phi^{UK} f_{vm}, \forall k \in \mathcal{K}, \forall u \in \mathcal{U}. \quad (20)$$

For each UAV u , its total amount of processed data cannot exceed its processing capacity. Mathematically,

$$\sum_{k=1}^K p_k(u) \leq C(u), \forall u \in \mathcal{U}. \quad (21)$$

Define $w_k(e_i) \geq 0$ as the amount of unprocessed flow on link e_i that belongs to the k -th source-destination pair. For the k -th source-destination pair, the amount of unprocessed flow on link e_i is

$$\sum_{e_i \in \delta_i^-(u)} w_k(e_i) = \max \left(\sum_{e_i \in \delta_i^+(u)} w_k(e_i) - p_k(u), 0 \right), \quad \forall k \in \mathcal{K}, \forall u \in \mathcal{U} \cup \mathcal{D}. \quad (22)$$

At each UAV, for each source-destination pair k , the total amount of incoming unprocessed flow is equal to the total amount of flow processed at this UAV plus the total amount of unprocessed outgoing flow. Formally, we have

$$\sum_{e_i \in \delta_i^-(u)} w_k(e_i) = p_k(u) + \sum_{e_i \in \delta_i^+(u)} w_k(e_i), \quad \forall k \in \mathcal{K}, \forall u \in \mathcal{U}. \quad (23)$$

Given demand M_k of source-destination pair k , the flow leaving source node s_k cannot exceed its demand. We have

$$\sum_{e_i \in \delta_i^+(s_k)} f_k(e_i) \leq M_k, \forall k \in \mathcal{K}. \quad (24)$$

For each source-destination pair k , the amount of unprocessed flow is less than or equal to the flow on a link, we have

$$w_k(e_i) \leq f_k(e_i), \forall k \in \mathcal{K}, \forall e_i \in \mathcal{E}_i. \quad (25)$$

In addition, for each source-destination pair k , the total amount of flow leaving a source node s_k must be unprocessed. Thus, we

²In practice, UAVs can be equipped with a Docker engine that runs containers or VNFs.

have

$$w_k(e_i) = f_k(e_i), \forall k \in \mathcal{K}, \forall e_i \in \delta_i^+(s_k). \quad (26)$$

Further, all data that arrives at a destination node must be processed. Formally,

$$w_k(e_i) = 0, \forall k \in \mathcal{K}, \forall e_i \in \delta_i^-(d_k). \quad (27)$$

G. UAV Energy Model

Without loss of generality, we consider three factors that affect the energy consumed by a UAV:

- *Deployment* – Define E_d^u (in Joule) as the energy consumed to deploy UAV u from a launch pad to the furthest *potential* location in $\mathcal{Z}$ and return to the launch pad. Let the corresponding distance be $d_{\max}$ (in meter). Let e (in joule/meter) be the energy consumption rate during flight. Then, E_d^u is calculated as

$$E_d^u = d_{\max} \times e. \quad (28)$$

- *Hovering* – Define E_h^u (in Joule) as the hovering energy of UAV u [31], which is computed as

$$E_h^u = (P_0 + P_i)t_h, \forall u \in \mathcal{U}, \quad (29)$$

where P_0 and P_i are constants depending on the weight of UAV, rotor disk area and air density [31]. The term t_h denotes the hovering time of UAVs.

- *Computation* – Define E_p^u (in Joule) as the energy consumed by UAV u to host Φ_u onboard VMs. As per [32], the computation energy of UAV u is

$$E_p^u = \sum_{\phi=1}^{\Phi_u} \kappa(f_{vm})^\xi t_p, \forall u \in \mathcal{U}, \quad (30)$$

where $\kappa \geq 0$ is the effective switched capacitance, ξ is a positive constant [33] and t_p is the processing time.

We note that as per [31], communication incurs orders of magnitude lower energy than hovering. Hence, this paper ignores communication related energy cost.

The total energy consumption E^u of UAV u is thus

$$E^u = E_d^u + E_h^u + E_p^u, \forall u \in \mathcal{U}. \quad (31)$$

Lastly, define E_0 as the initial energy of each UAV. To ensure UAV u has sufficient energy, we have

$$E^u \leq E_0. \quad (32)$$

IV. PROBLEM FORMULATION

We are now ready to formulate the problem for both splittable and non-splittable flow case. For both flow models, our aim is to obtain the best topology that maximizes the minimum amount of processed flow among all source-destination pairs. In general, we need to solve two problems: i) determining the amount of max-min computed flow for topology G_i and ii) searching for the best topology from $\mathcal{G}$. Note, maximizing the minimum computed flow rate leads to better fairness. Specifically, if we only maximize the computed flow rate, then some source-destination pairs may be neglected, i.e., their computed flow rate is zero.

The splittable flow case and the non-splittable flow case share a total of seven decision variables. These variables include: (a) topology selection α_i , (b) location selection $a_{u,i}^z$ of each UAV u , (c) link activation $I(e_i)$ of each p-link e_i , (d) virtual machine assignment X_ϕ^{UK} of each virtual machine, the amount of flow on each link $f_k(e_i)$, the amount of unprocessed flow on each link $w_k(e_i)$ and the amount of processed flow at each UAV $p_k(u)$. In addition, the non-splittable flow case requires one more decision variable, namely $S_k^i(p)$. This is because the problem in the non-splittable flow case requires path selection for each source-destination pair.

Define F_k^i as the total amount of processed flow of k -th source-destination pair in topology G_i . Formally, for each topology G_i , we have

$$F_k^i = \sum_{e_i \in \delta_i^-(d_k)} f_k(e_i), \forall k \in \mathcal{K}. \quad (33)$$

Let $\Lambda_i = \min\{F_k^i \mid k = 1, 2, \dots, K\}$ denote the minimum amount of computed flow among all source-destination pairs in topology G_i . For the splittable flow case, the problem is defined as a MILP, labeled as S-MILP. For ease of exposition, let $\mathbf{v} = \{\alpha_i, a_{u,i}^z, I(e_i), X_\phi^{UK}, f_k(e_i), w_k(e_i), p_k(u)\}$ be a set of decision variables. Mathematically, we have

$$\begin{aligned} \max_{\mathbf{v}} \quad & \sum_{i=1}^{|\mathcal{G}|} \alpha_i \Lambda_i \\ \text{s.t.} \quad & (1) - (15), \\ & (19) - (27). \end{aligned} \quad (34)$$

The problem in the non-splittable flow case is also formulated as a MILP, labeled as NS-MILP. Define $\mathbf{v}' = S_k^i(p) \cup \mathbf{v}$ as the set of decision variables. Formally, we have

$$\begin{aligned} \max_{\mathbf{v}'} \quad & \sum_{i=1}^{|\mathcal{G}|} \alpha_i \Lambda_i \\ \text{s.t.} \quad & (1) - (14), (16), (17), \\ & (19) - (27). \end{aligned} \quad (35)$$

A. Remarks

First, we do not consider trajectory planning as our aim is to determine the optimal location of each UAV or a network topology that maximizes objective (34) or (35) for a set of source-destination pairs with a *fixed* location. Note, if UAVs move after placement, then the resulting topology may yield a non-optimal objective value. Further, if a source or destination moves, then our solutions can be used to change the position of UAVs accordingly.

Second, our formulations and solutions assume fixed link capacities, and do not consider time-varying channel conditions or errors. Hence, our formulations and solutions consider the ideal case and serve as a theoretical upper bound that can be used to compare future solutions that consider more realistic channel conditions.

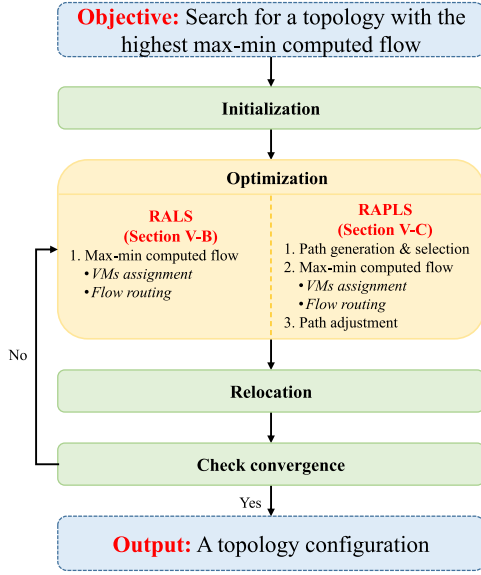


Fig. 4. General steps of proposed heuristic solutions. The yellow box for the optimization step contains the key ideas of RALS and RAPLS, respectively.

Third, the problem is challenging as the total number of topologies, i.e., size of $|\mathcal{G}|$, increases exponentially with the number of UAVs. Moreover, in the non-splittable flow case, the solution space is $\binom{P_k}{K}$ times larger than in the splittable flow case. This is because each source-destination pair needs to select a path in each topology G_i . We also note that in the non-splittable flow case, given a topology, a special case of our problem is equivalent to the maximum flow problem with integral flow, a strongly NP-hard problem [34]. Therefore, in the following two sections, we will introduce two heuristic algorithms to construct a topology for both flow models.

Lastly, we note that our solutions are used by an operator to compute a solution offline. Given the set of decision variables $\mathbf{v}$ and $\mathbf{v}'$ for problems (34) and (35), respectively, an operator can then instruct UAVs to hover at their computed location, install routes and VMs, and sources are informed to emit a given amount of traffic.

V. SOLUTIONS

We will introduce two heuristic solutions, namely, Resource-Aware Location Selection (RALS) for the splittable flow case, and Resource-Aware Path and Location Selection (RAPLS) for the non-splittable flow case. Fig. 4 provides an overview of these solutions. They have four key steps: i) *Initialization*, which creates an initial topology, ii) *Optimization*, which aims to optimize the max-min computed flow of a given topology, iii) *Relocation*, which aims to create a new topology by relocating UAVs, and iv) *Check convergence*, which checks whether to stop searching for a new topology. Note, both solutions share the same *Initialization*, *Relocation* and *Check convergence* steps. The main difference is their *Optimization* step, where for RALS, it simply optimizes the max-min computed flow of a topology.

Algorithm 1: Pseudocode for *InitialTopo()*.

Input: $\mathcal{V}, \mathcal{Z}$
Output: $\hat{G}$

```

1  $\hat{G} = \emptyset$ 
2  $u = 1$ 
3  $\text{Loc}(u) = \text{Random}(\mathcal{Z})$ 
4  $\hat{G} = \hat{G} \cup \text{Loc}(u)$ 
5 for  $u \leftarrow 2$  to  $|\mathcal{U}|$  do
6    $\mathcal{N}_u = \mathcal{L}_{u-1}^R$ 
7    $\text{Loc}(u) = \text{Random}(\mathcal{N}_u)$ 
8    $\hat{G} = \hat{G} \cup \text{Loc}(u)$ 
9 end
10 Return  $\hat{G}$ 

```

On the other hand, RAPLS also optimizes the path of source-destination pairs. Next, we will first discuss the said common steps. After that, we present the *Optimization* step of RALS and RAPLS.

A. Common Steps

1) *Initialization*: The main idea is to assign each UAV a location in turn. For example, it first assigns a location to UAV with index u followed by $u + 1$, and so forth. Specifically, when assigning the location of UAV $u + 1$, the location must be selected from a set of *potential* locations that are within the transmission range of u . To illustrate this, define $\mathcal{L}_u^R \in \mathcal{Z}$ as the set of *potential* locations within the transmission range of UAV u , where R is the transmission range of all UAVs. Define $\mathcal{N}_u$ as the set of potential locations of UAV u . We have

$$\mathcal{N}_u = \mathcal{Z}, \quad u = 1, \quad (36a)$$

$$\mathcal{N}_u = \mathcal{L}_{u-1}^R, \quad \forall u \in \{2, 3, \dots, U\}. \quad (36b)$$

We assume each UAV u can select from N *potential* locations, which are randomly drawn from $\mathcal{N}_u$. Let $\text{Loc}(u)$ denote the assigned location of UAV u . When a UAV has an assigned location, it constructs communication links with other UAVs, as well as builds the initial topology $\hat{G}$.

The aforementioned steps are implemented by *InitialTopo()*, see Algorithm 1. It takes as inputs the set of nodes $\mathcal{V}$ and the set of *potential* locations $\mathcal{Z}$. For the first UAV, it is randomly assigned a location from $\mathcal{Z}$, see lines 1-1. Then in lines 1-1, for all subsequent UAVs, each UAV u is assigned a location that is within the transmission range of UAV $u - 1$. It terminates when all UAVs have a location.

2) *Relocation*: The main idea is to identify the UAV, say $\hat{u}$, with the most available processing capacity and then update its location. Specifically, define set $\hat{\mathcal{R}}_i = \{\hat{r}_u^i \mid u = 1, 2, \dots, U\}$ to contain the residual processing capacity of all UAVs in topology G_i . Mathematically, we have

$$\hat{u} = \arg \max_{u \in \mathcal{U}} \hat{r}_u^i. \quad (37)$$

After that, we select a location in $\mathcal{N}_{\hat{u}}$ that will result in the most number of neighbors. To do this, let $\mathcal{H}(n_{\hat{u}})$ denote the node degree of potential location $n_{\hat{u}}$, where $n_{\hat{u}} \in \mathcal{N}_{\hat{u}}$. Note that for a

Algorithm 2: Pseudocode for RALS.

Input: $\mathcal{V}, \lambda, \Delta$
Output: G_s

```

1  $G_s = \emptyset, \hat{G} = \text{InitialTopo}(\mathcal{V}), \Lambda_{\hat{G}} = \text{S-MILP}(\hat{G})$ 
2 for  $t \leftarrow 1$  to  $\lambda$  do
3    $\hat{u} = \text{MaxResource}(\mathcal{U})$ 
4    $G' = \text{Relocate}(\hat{u})$ 
5    $\Lambda_{G'} = \text{S-MILP}(G')$ 
6   if  $\sum_{k=1}^K p_k(\hat{u}) == C(\hat{u})$  then
7     Break
8   end
9   if  $\text{CheckConverge}(\Delta) == \text{True}$  then
10    Break
11  end
12 end
13  $G_s = G'$ 
14 Return  $G_s$ 

```

UAV u , we define the node degree of a potential location n_u as the number of neighbors of n_u if it is selected by UAV u . Thus, we have

$$\text{Loc}(\hat{u}) = \arg \max_{n_{\hat{u}} \in \mathcal{N}_{\hat{u}}} \mathcal{H}(n_{\hat{u}}), \quad (38)$$

where $\text{Loc}(\hat{u})$ is the updated location of $\hat{u}$. This location is then included in the current topology to construct communication links with other nodes to finalize the Relocation step.

3) *Check Convergence*: If the computed results in the past Δ iterations remain the same, then we have convergence.

B. Rals

The main idea is to minimize unused processing capacity at UAVs. Algorithm 2 presents RALS. It requires the set of nodes $\mathcal{V}$, the maximum number of iterations λ , and the value of Δ as inputs to generate topology G_s . Note that $\mathcal{V} = \mathcal{S} \cup \mathcal{D} \cup \mathcal{U}$. Referring to Algorithm 2, in line 2, RALS assigns the initial location of UAVs and computes the max-min computed flow value for the initial topology $\hat{G}$ by solving S-MILP (34). In lines 2-2, it reassigns the location of the UAV with the highest available computing resource. After that, in line 2, it solves S-MILP (34) to obtain the updated max-min computed flow. Line 2 checks the residual capacity of $\hat{u}$. If UAV $\hat{u}$ has no available processing capacity, RALS concludes that the current topology has the highest max-min computed flow, and terminates. Otherwise, if $\hat{u}$ has residual processing capacity but the last Δ iterations have the same computed flow result, RALS terminates, see line 2.

C. Rapls

It has two additional steps as compared to RALS. First, for a constructed topology, RAPLS generates up to P_k paths for each source-destination pair k and then it selects the shortest path for each source-destination pair. The selected path of all source-destination pairs is stored in set $\hat{P}$. Note that the reason for selecting the shortest path for each source-destination pair is to improve the computational efficiency of RAPLS. This is

because for a source-destination pair, the shortest path contains the minimum number of UAVs. Fewer UAVs will lead to fewer VMs. Fewer VMs also means there are fewer constraints of type (19)–(20), which helps reduce the program size or computational complexity of NS-MILP (35).

Second, after calculating the max-min computed flow for a topology, RAPLS inspects the amount of computed flow for each source-destination pair. RAPLS then selects the source-destination pair with the minimum amount of computed flow and adjusts its selected path.

Algorithm 3 presents the main steps of RAPLS. It takes as inputs the set of nodes $\mathcal{V}$, the maximum number of iteration λ , the check convergence parameter Δ , and the maximum number of paths for each source-destination pair k , which it labels as P_k . Referring to Algorithm 3, in line 3, the function $\text{AdjustPath}(\hat{G})$ retrieves the current topology information as well as the computed flow value of each source-destination pair. It updates the path of the source-destination pair with the minimum computed flow value. The updated path is denoted as $\check{P}$. Specifically, the function $\text{AdjustPath}(\hat{G})$ randomly selects a path from the set of available paths P_k . After that in line 3, RAPLS calculates the max-min computed flow again by solving NS-MILP (35) with the updated path $\check{P}$. If the path adjustment step improves the max-min computed flow value, RAPLS will repeat this step for the next iteration $t + 1$, see lines 3-3. In each iteration, if the path adjustment step fails to improve the max-min computed flow value, RAPLS will follow the same steps as RALS, where it updates the location of the UAV with the highest residual processing capacity. Note that in line 3, RAPLS returns the set of selected paths as the final topology. This is because in the non-splittable flow case, an active topology is represented by a set of selected paths of all source-destination pairs.

D. Discussion

RALS and RAPLS are greedy heuristics. Note that RALS applies S-MILP (34), and RAPLS applies NS-MILP (35) in each iteration. However, RALS and RAPLS do not maintain the collection of topologies $\mathcal{G}$. This means RALS and RAPLS do not guarantee the best topology configuration that has the max-min computed flow. The reason is that RALS and RAPLS respectively apply S-MILP (34) and NS-MILP (35) to obtain the max-min computed flow for a single topology, rather than a set of topologies.

VI. EVALUATION

We conducted our experiments in Python with Gurobi, and consider both the splittable and non-splittable flow case. There are two sets of experiments carried out in small and large networks. A small network has $U \leq 7$ UAVs and $K \leq 5$ source-destination pairs. A large network contains $U \geq 8$ UAVs and $K \geq 6$ source-destination pairs. Note that it is computationally intractable to compute the optimal solution in large networks. For both flow models, the optimal solution is labeled as S-MILP and NS-MILP, respectively. Their corresponding heuristic solutions are respectively called RALS and RAPLS. We benchmark against the following solutions:

Algorithm 3: Pseudocode for RAPLS.

Input: $\mathcal{V}, \lambda, \Delta, P_k$
Output: G_{ns}

```

1  $G_{ns} = \emptyset, \hat{G} = \text{InitialTopo}(\mathcal{V}), \hat{P} = \text{FindPath}(\hat{G}),$ 
   $\Lambda_{\hat{P}} = \text{NS-MILP}(\hat{P})$ 
2 for  $t \leftarrow 1$  to  $\lambda$  do
3    $\tilde{P} = \text{AdjustPath}(\hat{G})$ 
4    $\Lambda_{\tilde{P}} = \text{NS-MILP}(\tilde{P})$ 
5   if  $\Lambda_{\tilde{P}} > \Lambda_{\hat{P}}$  then
6      $\tilde{P} = \hat{P}$ 
7      $\Lambda_{\hat{P}} = \Lambda_{\tilde{P}}$ 
8     Continue
9   end
10   $\hat{u} = \text{MaxResource}(\mathcal{U})$ 
11   $G' = \text{Relocate}(\hat{u})$ 
12   $\hat{P} = \text{FindPath}(G')$ 
13   $\Lambda_{\hat{P}} = \text{NS-MILP}(\hat{P})$ 
14  if  $\sum_{k=1}^K p_k(\hat{u}) == C(\hat{u})$  then
15    Break
16  end
17  if  $\text{CheckConverge}(\Delta) == \text{True}$  then
18    Break
19  end
20 end
21  $G_{ns} = \hat{P}$ 
22 Return  $G_{ns}$ 

```

- *RALS-Random (RALS-R)*: This is a variation of RALS, where in each iteration, the UAV with the most available processing capacity randomly selects a potential location.
- *RAPLS-Random (RAPLS-R)*: This is a variation of RAPLS, where in each iteration, RAPLS-R does not adjust the path for the source-destination pair with the minimum computed flow rate value. Instead, the solver randomly updates the location of the UAV with the most residual processing capacity.

UAVs operate in a square area with size $100 \times 100 \text{ m}^2$. The grid contains 20×20 grid lines, which means the total number of potential locations on the grid is $|\mathcal{Z}| = 400$. Except for Section VI-E, we set the transmit power of UAVs to $P_t = 20$ dBm. The transmission range of UAVs, i.e., R , is drawn from the range $[50, 70]$ (in meters). Each UAV is equipped with a 5000 mAh battery, which is equivalent to $E_0 = 227200$ joules [35]. For deployment energy, the flight energy consumption rate e is set to 224 Joule/meter [36]. As per [31], constants P_0 and P_i , which relate to UAV hovering energy, are set to 79.8 and 88.6, respectively. The hovering time of UAVs is $t_h = 10$ minutes. In terms of processing energy consumption, we set $\kappa = 10^{-26}$ and $\xi = 3$ [33]. According to [37], the maximum operating frequency of a VM is 5 GHz, and the VM processing time t_p is set to one second. Given these parameters, the maximum number of VMs, namely Φ_u , is 20. Except for the experiment in Section VI-D, we set the operating frequency of each VM to $f_{vm} = 1$ GHz. We set the maximum number of iterations λ of RALS and RAPLS to 100. To check for convergence, we set

TABLE III
SIMULATION PARAMETERS

Parameter	Value
Area size	$100 \times 100 \text{ m}^2$
Number of source-destination pairs	1 to 5
Number of UAVs	1 to 7
Number of VMs per UAV Φ_u	1 to 20
UAV flying altitude h	30 m
Transmit power P_t	20 to 30 dBm
Transmission range of UAVs R	50 to 70 m
UAV battery capacity [35]	5000 mAh
System bandwidth W [39]	20 MHz
Carrier frequency f [38]	2.4 GHz
CPU frequency of each VM f_{vm} [37]	0.2 to 5.0 GHz
CPU cycles to process one bit [30]	120
Speed of light c	$3 \times 10^8 \text{ m/s}$
Maximum number of iteration λ	100
Convergence threshold Δ	10
SNR threshold β [38]	12 dB
Ambient noise power N_0 [40]	-90 dBm
Path loss exponent γ [24]	2
Excessive LoS path loss η_{LoS} [24]	1.0
Excessive NLoS path loss η_{NLoS} [24]	20

Δ to 10. We note that the average convergence time for RALS and RAPLS is 45 and 62 iterations, respectively. The number of *potential* locations of each UAV for RALS and RAPLS is set to $N = 10$ for the sake of balancing computation load and performance. We set the SNR threshold β to 12 dB [38]. As per [24], the path loss exponent γ is two, and the excessive path loss for LoS and NLoS condition is $\eta_{LoS} = 1.0$ and $\eta_{NLoS} = 20$, respectively. Our results are an average of 100 simulation runs. In addition, we have presented the 95% confidence interval for each curve as a colored band. Table III lists parameter values.

A. UAVs Density

The number of source-destination pairs is set to two and each UAV is equipped with four VMs. Fig. 5(a) shows the relationship between the number of UAVs and the amount of max-min computed flow of the splittable flow case. The flow value for S-MILP improves from 12.53 to 141.61 Mbps, and the result of RALS increased from 12.32 to 130.04 Mbps. All algorithms have a higher amount of max-min computed flow with more UAVs. When the number of UAVs increases from one to seven, the number of available VMs increases from four to 28, which results in a higher processing capacity. In addition, the number of links between UAVs increases, and the distance between UAVs decreases, which results in higher link capacity. Moreover, in the splittable flow case, traffic can be routed across any path, which means the utilization of VMs and links is maximized. Note that RALS yields a max-min computed flow value that is nearly 81% of S-MILP. This is because in each iteration RALS identifies the UAV with the most available processing capacity and shifts its location to make full use of its processing capacity. The gap between RALS and RALS-R is around 25%. The reason is that when updating the location of a selected UAV, RALS will select a potential location with the most number of neighboring UAVs. As for RALS-R, during the location update phase, it only randomly selects a potential location.

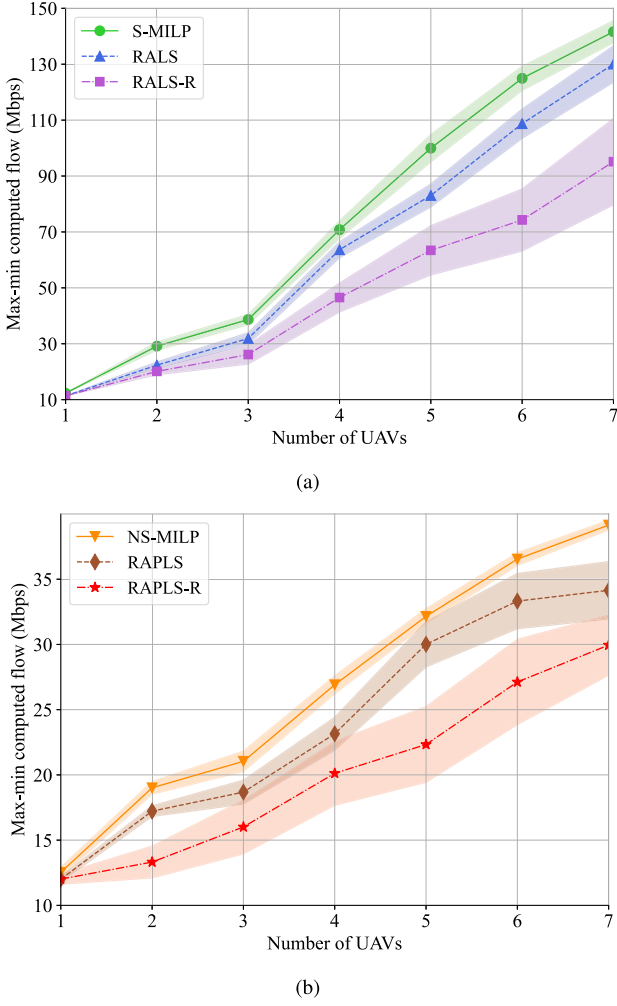


Fig. 5. Number of UAVs versus the amount of max-min computed flow of (a) splittable flow case and (b) non-splittable flow case.

Fig. 5(b) presents the result of the non-splittable flow case. The amount of max-min computed flow of NS-MILP increased from 12.32 to 39.15 Mbps, and the amount of computed flow of RAPLS increased from 11.36 to 34.15 Mbps. This is because when the density of UAVs increases, UAVs are closer to each other, which helps increase link capacity. Further, in the non-splittable flow case, traffic is routed on one selected path. On one hand, not all UAVs are guaranteed to be involved in the selected path, thus resulting in unused UAVs. On the other hand, even if all UAVs form a linear topology, there will be redundant processing capacity that can not be used because of a bottleneck link in the selected path. In addition, the performance of RAPLS-R is 22% lower than RAPLS on average. This is because RAPLS-R is not aware of bottleneck links. That is, in each iteration, RAPLS-R only shifts the location of the UAV with the most residual processing capacity. However, in the non-splittable flow case, a bottleneck link is the main reason that affects the amount of max-min computed flow. In this regard, RAPLS-R has a higher probability of encountering the same

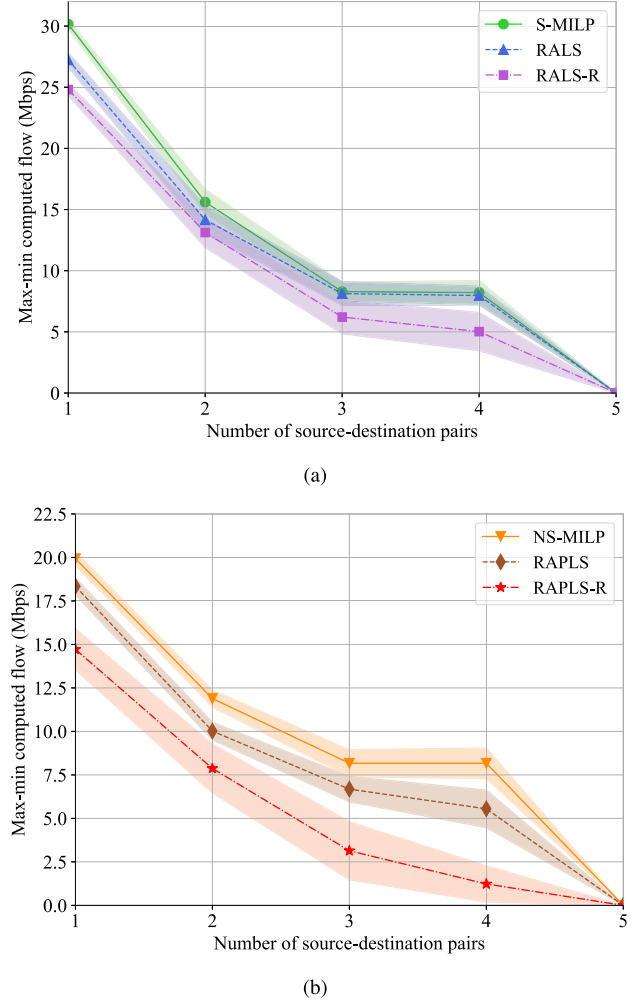


Fig. 6. Number of source-destination pairs versus the amount of max-min computed flow of (a) splittable flow case and (b) non-splittable flow case.

bottleneck link that blocks the amount of max-min computed flow from increasing after a location update.

B. Number of Source-Destination Pairs

We consider one VM per UAV and a total of four UAVs. Referring to Fig. 6(a) for the splittable flow case, the amount of max-min computed flow of S-MILP, RALS and RALS-R decreases with more source-destination pairs. This is because there are more source-destination pairs that share VMs. Note that when $K = 5$, the amount of max-min computed flow is zero for all three solutions. This is because the total number of VMs is four and each VM can only serve one source-destination pair. Thus, there will be at least one pair that is not assigned a VM. As a result, the amount of max-min computed flow is zero. The amount of max-min computed flow is constrained by VMs processing capacity. Specifically, in this experiment, the processing capacity of a VM is 8.33 Mbps, which is precisely the amount of computed flow of S-MILP when $K = 3$ and $K = 4$. This is because the minimum number of VMs assigned to one pair is one and the average link capacity is higher than the

processing capacity of VMs. Moreover, RALS and RALS-R achieve 98% and 80% of the amount of max-min computed flow attained by S-MILP. This is because there are only four UAVs and the topology search space is relatively small and both approaches achieve near-optimal performance.

Next, we study the non-splittable flow case. Referring to Fig. 6(b), the amount of max-min computed flow follows the same trend as the splittable flow case with more source-destination pairs. The performance of RAPLS is 8% and 21% lower than NS-MILP when $K = 1$ and 4, respectively. This is because the number of paths increased proportionally with K . As a result, path diversity increases for each source-destination pair when K becomes larger. This means there is a higher probability that RAPLS achieves a poor result when K is large. Moreover, the gap between RAPLS and RAPLS-R increased from 20% to 76% when K increased from one to four. This is because RAPLS-R only updates the location of a selected UAV without updating a selected path. Given that it does not adjust any paths, there is a higher probability of bottleneck links or UAVs, which lowers the performance of RAPLS-R.

C. Number of VMs Per UAV

The third experiment investigates varying number of VMs on each UAV, i.e., Φ_u . We increase Φ_u from one to 20. The number of source-destination pairs is set to four and the number of UAVs is set to three. Referring to Fig. 7(a), additional VMs lead to a higher amount of max-min computed flow for S-MILP, RALS, and RALS-R. RALS and RALS-R achieve on average 83% and 60% of S-MILP, respectively. When $\Phi_u = 1$, the total number of VMs is less than the total number of source-destination pairs. Thus the amount of max-min computed flow is zero. For S-MILP, this amount is increased by 275% as the number of VMs per UAV increases from two to eight. As for RALS, this amount increased by 16.95 Mbps, i.e., from 8.05 to 25 Mbps. This is because each source-destination pair is assigned more VMs to compute incoming flows. For example, when $\Phi_u = 2$, there will be one source-destination pair that is assigned one VM. However, when $\Phi_u = 3$, the minimum number of VMs assigned to each pair is two, which doubles the amount of processed flow of each source-destination pair. Further, when $\Phi_u = 4$, the max-min computed flow value can be compared against Fig. 5(a) in the case when $U = 3$. In Fig. 5(a), when $U = 3$, each source-destination pair is assigned six VMs, and S-MILP achieves a max-min computed flow value of 40 Mbps, which is twice as much as in Fig. 7(a) when $\Phi_u = 4$. This is because in this experiment, there are four source-destination pairs, and when $\Phi_u = 4$, each source-destination pair is only assigned three VMs. Thus, the max-min computed flow is around 22 Mbps, which is around half of the value of Fig. 5(a) when $U = 3$. In addition, for S-MILP, RALS and RALS-R, the amount of max-min computed flow remains unchanged when Φ_u increases from eight to 20. This indicates that the amount of max-min computed flow is bounded by link capacity.

As for the non-splittable flow case, Fig. 7(b) shows that NS-MILP has a higher amount of max-min computed flow with more VMs. Specifically, increasing the number of VMs

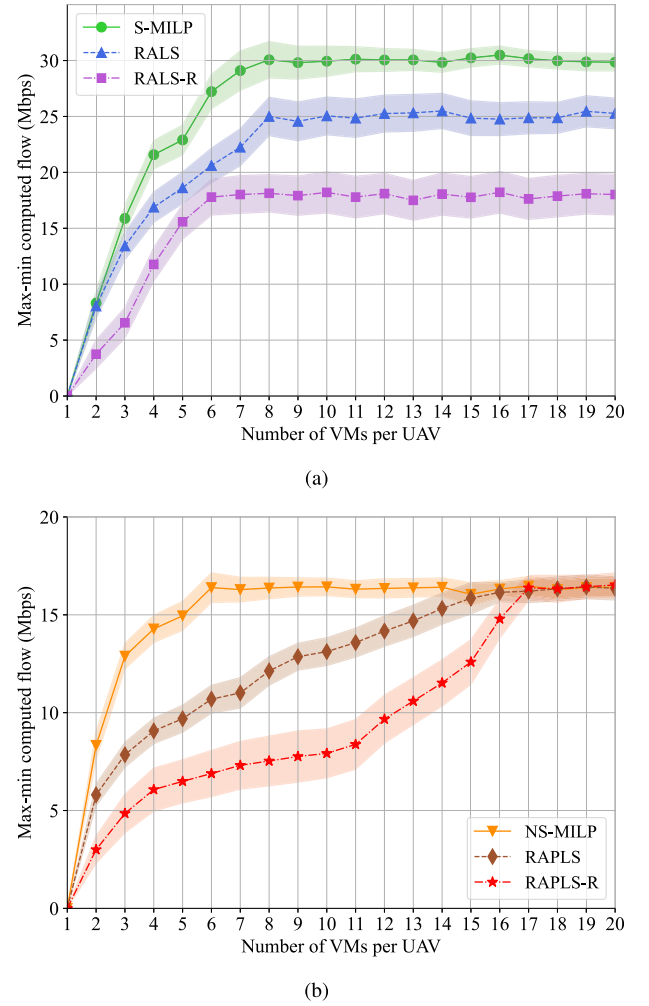


Fig. 7. Number of VMs per UAV versus the amount of max-min computed flow of (a) splittable flow case and (b) non-splittable flow case.

per UAV from one to six results in a significant increment in the amount of max-min computed flow, i.e., from zero to 17.01 Mbps. This is because a higher number of VMs indicates a higher UAVs processing capacity, which means one UAV can support multiple source-destination pairs without experiencing a bottleneck. When the value of Φ_u increases from six to 20, NS-MILP remains at the same amount of computed flow value. This indicates that the amount of max-min computed flow value is limited by link capacity. Another observation is that RAPLS and RAPLS-R reached a higher amount of max-min computed flow as Φ_u is larger and their average max-min computed flow is within 18% and 42% of NS-MILP. This is because a higher value of Φ_u prevents processing capacity bottleneck when different source-destination pairs share the same UAV. Unlike the experiment shown in Fig. 5(b), increasing the number of VMs will not result in unused UAVs. Furthermore, both RAPLS and RAPLS-R achieve the optimal amount of max-min computed flow when the value of Φ_u is greater than 16. This is because, in this experiment setup the average link capacity is approximately 17 Mbps, which means different path selections or UAVs

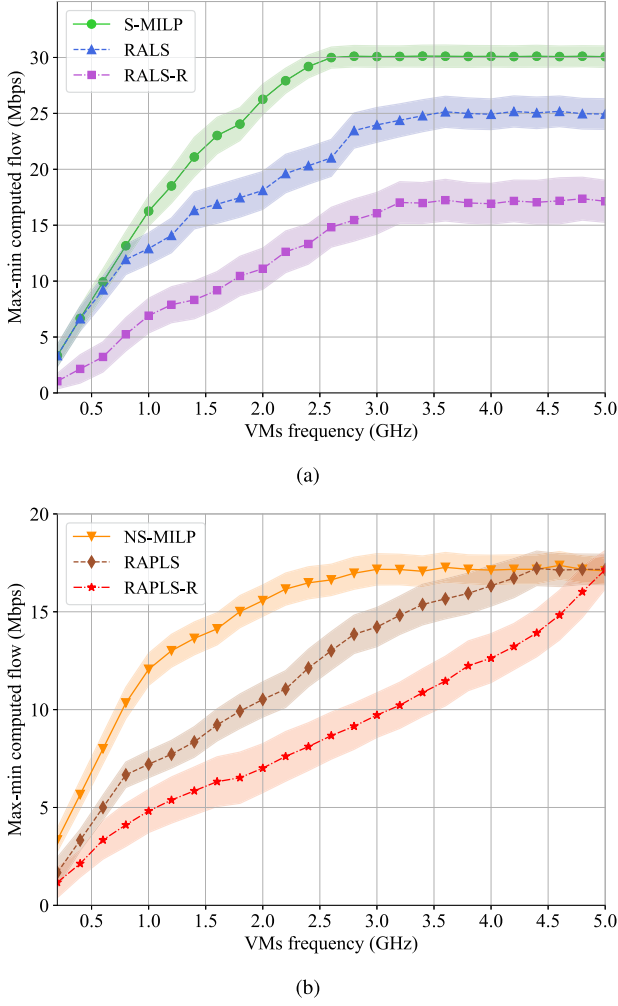


Fig. 8. CPU frequency of VMs versus the amount of max-min computed flow of (a) splittable flow case and (b) non-splittable flow case.

location selections will be limited by the same bottleneck link capacity.

D. VMs Frequency

We now vary the CPU frequency of VMs between 0.2 and 5 GHz. This experiment considers $K = 4$, $\Phi_u = 3$ and a total of three UAVs. Referring to Fig. 8(a), the amount of max-min computed flow of S-MILP, RALS and RALS-R continues to rise when f_{vm} increases up to 3 GHz. This is expected as the resulting network continues to take advantage of the higher processing capacity at UAVs. After that, the amount of max-min computed flow value remains stable for all three solutions. This is because the limited link capacity prevents the amount of computed flow from further increasing. RALS-R achieves 56% of the amount of processed flow of S-MILP and RALS achieves 84% of S-MILP. This is because RALS-R and RALS have different UAVs location update policies that affect link capacity. Specifically, during the location update step, RALS-R randomly selects a potential location for a UAV, while RALS selects a potential location with the most neighbors.

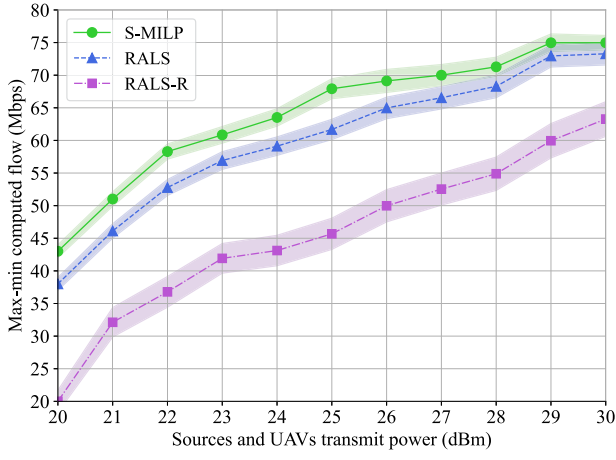
Referring to Fig. 8(b), both RAPLS and RAPLS-R reach the optimal amount of computed flow value as f_{vm} increases from 0.2 to 5.0 GHz. When $f_{vm} \leq 5.0$, RAPLS and RAPLS-R are compared to NS-MILP, and the average performance gap is 15% and 35%, respectively. This is because the main bottleneck gradually switched from UAVs processing capacity to link capacity when we increased the CPU frequency of VMs. Unlike the scenario in the splittable flow case, increasing UAVs processing capacity helps RAPLS and RAPLS-R reach the optimal amount of max-min computed flow value. This is because in the non-splittable flow case, there is a high probability that different source-destination pairs need to share the same UAV to construct a path. In this regard, when $f_{vm} \geq 3$ GHz, a UAV is able to process the incoming flow from all pairs without causing a processing capacity bottleneck. Further, despite the different path and UAV location update policies, both RAPLS and RAPLS-R will fully utilize link capacity. As a result, both RAPLS and RAPLS-R are able to achieve the optimal amount of computed flow.

Note, compared to the experiment in Section VI-C, increasing VMs frequency is computationally more efficient than increasing the number of VMs, i.e., fewer constraints required and less computation time. Specifically, when the value of Φ_u is fixed, higher VMs frequency does not require the solver to execute VMs assignment again. Thus, for all solutions in both flow models, the solver obtains the amount of max-min computed flow quicker, i.e., three times faster.

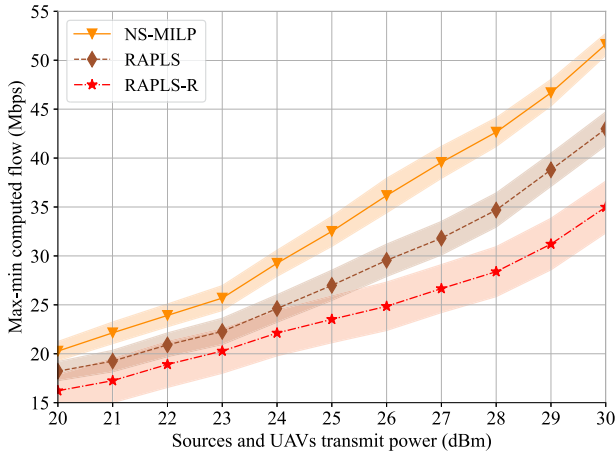
E. Transmit Power of Sources and UAVs

In this experiment, we gradually increase the transmit power of sources and UAVs when $K = 4$, $U = 3$ and $\Phi_u = 7$. As per Fig. 9(a), the amount of max-min computed flow increases when P_t rises from 20 to 30 dBm. This is because when UAVs have a larger transmit power, all active links will correspondingly have higher capacity. In addition, because the SNR threshold β is fixed at 12 dB, the transmission range also increases when P_t increases. As a result, there will be more active links in the network. S-MILP is able to find the optimal UAVs location that results in the highest link capacity and thus has the highest performance. We observe that the amount of computed flow stops increasing when $P_t \geq 29$ dBm. The reason is that the network bottleneck shifts from link capacity to UAVs processing capacity. RALS will select a potential location that results in most neighboring UAVs. Note that higher transmit power results in more links. Thus, RALS achieves on average 87% of the amount of max-min computed flow achieved by S-MILP.

Referring to Fig. 9(b), the amount of max-min computed flow is proportional to P_t . This is because a larger P_t results in a higher link capacity. Moreover, in the non-splittable flow case, traffic flow is routed on a single path. In this regard, a source can send more data to its corresponding destination for processing, which results in a higher amount of max-min computed flow of the network. An observation is that the amount of computed flow achieved by RAPLS-R is within 9% and 21% of RAPLS when P_t is 20 and 30 dBm, respectively. This is because when $P_t \leq 25$ dBm, UAVs processing capacity is able to support the



(a)



(b)

Fig. 9. Sources and UAVs transmit power versus the amount of max-min computed flow of (a) splittable flow case and (b) non-splittable flow case.

higher link capacity. In this regard, both RAPLS-R and RAPLS are able to take advantage of the higher link capacity. On the other hand, when $P_t \geq 26$ dBm, there is a higher probability that UAVs lead to a processing bottleneck. RAPLS-R does not have a path adjustment step to avoid a processing bottleneck or a link bottleneck, thus the gap between RAPLS-R and RAPLS grows.

F. Large Networks

In this section, we omit results from S-MILP and NS-MILP because they are computationally intractable. We study i) the number of UAVs U for the splittable flow case, where we consider $K = 3$, $\Phi_u = 4$ and increase U from five to 50, and ii) number of pairs K for non-splittable flow case, where we consider $U = 5$, $\Phi_u = 6$ and increase K from five to 35.

1) *Splittable flow*: According to Fig. 10, the amount of max-min computed flow of RALS-R has a 214 Mbps increment when the number of UAVs increases from five to 50. As for RALS, the average amount of max-min computed flow reaches 350 Mbps from 40 Mbps. The reason is that additional UAVs create more

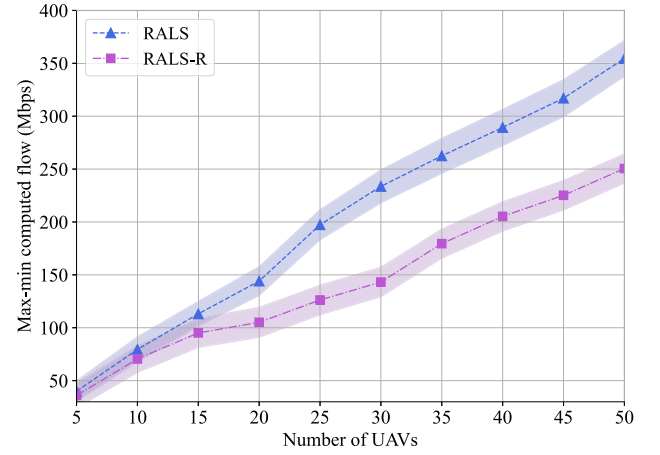


Fig. 10. Max-min computed splittable flow with the varying number of UAVs in large networks.

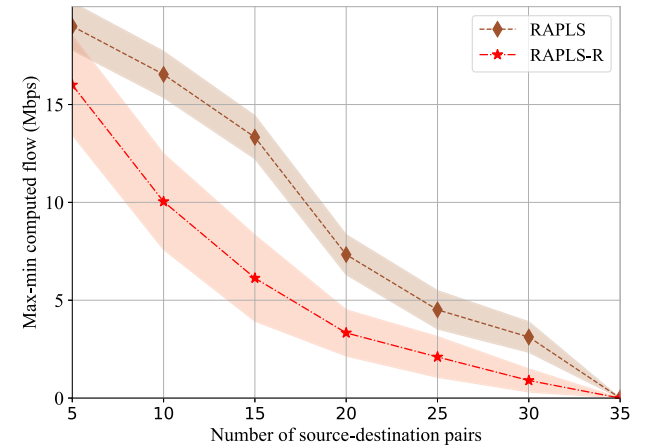


Fig. 11. Max-min non-splittable computed flow with the varying number of source-destination pairs in large networks.

connections as well as reduce the distance between any two UAVs. Further, more UAVs provide additional processing capacity. As a result, each source-destination pair is able to send more data. Note that when $U = 50$, RALS-R achieves 70% of the amount of computed flow of RALS. The reason for the 30% gap is that RALS-R applies a random scheme to update the location of a UAV. The updated location has a higher probability to suffer from insufficient link capacity, which blocks the amount of max-min computed flow from further increasing. However, RALS updates the location of a UAV by maximizing the number of neighbors. Although RALS does not guarantee the optimal location of UAVs, it is able to find a location that results in higher link capacity as compared to RALS-R. Moreover, because of the difference in the Relocation step, RALS-R requires 30% less execution time as compared to RALS.

2) *Non-splittable flow*: Referring to Fig. 11, the amount of max-min computed flow decreases as expected when K increases to 30. For example, the amount of max-min computed flow decreases by 80% from 18.67 Mbps. This is reasonable as the number of VMs is fixed and VMs are shared by more

source-destination pairs. In addition, the total number of VMs is 30, which means when $K \geq 30$, the amount of max-min computed flow will be zero. Moreover, the max-min computed flow achieved by RAPLS-R is always lower than RAPLS. This is because RAPLS-R does not adjust the selected path of each pair, which means RAPLS-R has a higher probability to cause link congestion or UAVs to saturate their processing capacity. On average, RAPLS-R achieves 71% of the amount achieved by RAPLS. In addition, RAPLS-R requires around 45% less execution time as compared to RAPLS because RAPLS-R does not have the path adjustment step.

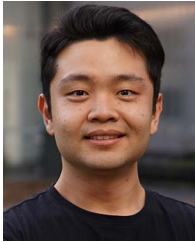
VII. CONCLUSION

This paper studies the max-min computed flow problem in a network that applies UAVs as *flying middleboxes*. Both splittable flow and non-splittable flow cases are considered. It outlines two MILPs namely S-MILP and NS-MILP that jointly optimizes the location of UAVs, link activation, VMs assignment as well as traffic routing between all source-destination pairs. Accordingly, it also introduces two heuristic algorithms called RALS and RAPLS, respectively for each flow model. Both RALS and RAPLS do not require knowledge of all potential locations of UAVs. Further, RAPLS does not require knowledge of all combinations of potential paths. Both heuristic algorithms reduce the search space and are able to obtain a max-min computed flow value within a reasonable time. Specifically, RALS and RAPLS achieve 45% and 53% less computation time against S-MILP and NS-MILP, respectively. Our simulation results show that RALS achieves on average 83% of the amount of computed flow compared to S-MILP. The amount of processed flow of RAPLS is within 20% of NS-MILP. Possible future works include developing distributed algorithms for both flow models to further optimize the VMs assignment, and consider the energy evolution of UAVs during data computing and transmitting. A possible future work is to assume the case whereby UAVs operate on the same frequency. In this respect, a sub-problem is to consider joint routing and link scheduling.

REFERENCES

- [1] R. Mijumbi, J. Serrat, J.-L. Gorricho, N. Bouten, F. De Turck, and R. Boutaba, "Network function virtualization: State-of-the-art and research challenges," *IEEE Commun. Surv. Tut.*, vol. 18, no. 1, pp. 236–262, First Quarter, 2016.
- [2] J. Sherry, S. Hasan, C. Scott, A. Krishnamurthy, S. Ratnasamy, and V. Sekar, "Making middleboxes someone else's problem: Network processing as a cloud service," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 42, pp. 13–24, Aug. 2012.
- [3] Z. Xu, W. Gong, Q. Xia, W. Liang, O. F. Rana, and G. Wu, "NFV-enabled IoT service provisioning in mobile edge clouds," *IEEE Trans. Mobile Comput.*, vol. 20, no. 5, pp. 1892–1906, May 2021.
- [4] X. Long, B. Liu, F. Jiang, Q. Zhang, and X. Zhi, "FPGA virtualization deployment based on docker container technology," in *Proc. Int. Conf. Mech. Control Comput. Eng.*, Harbin, China, 2020, pp. 473–476.
- [5] I. Bekmezci, O. K. Sahingoz, and C. Temel, "Flying ad-hoc networks (FANETs): A survey," *Ad Hoc Netw.*, vol. 11, pp. 1254–1270, May 2013.
- [6] Y. Li and L. Cai, "UAV-assisted dynamic coverage in a heterogeneous cellular system," *IEEE Netw.*, vol. 31, pp. 56–61, Jul./Aug. 2017.
- [7] N. Zhao, Z. Ye, Y. Pei, Y.-C. Liang, and D. Niyato, "Multi-agent deep reinforcement learning for task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 6949–6960, Mar. 2022.
- [8] B. Nogales, V. Sanchez-Aguero, I. Vidal, F. Valera, and J. Garcia-Reinoso, "A NFV system to support configurable and automated multi-UAV service deployments," in *Proc. 4th ACM Workshop Micro Aerial Veh. Netw. Syst. Appl.*, Munich, Germany, 2018, pp. 39–44.
- [9] A. Hermosilla, A. M. Zarca, J. B. Bernabe, J. Ortiz, and A. Skarmeta, "Security orchestration and enforcement in NFV/SDN-aware UAV deployments," *IEEE Access*, vol. 8, pp. 131779–131795, 2020.
- [10] R. Cziva, C. Anagnostopoulos, and D. P. Pazaros, "Dynamic, latency-optimal VNF placement at the network edge," in *Proc. IEEE Conf. Comput. Commun.*, Honolulu, HI, USA, 2018, pp. 693–701.
- [11] G. Sallam and B. Ji, "Joint placement and allocation of virtual network functions with budget and capacity constraints," in *Proc. IEEE Conf. Comput. Commun.*, Paris, France, 2019, pp. 523–531.
- [12] P. Mandal, "Comparison of placement variants of virtual network functions from availability and reliability perspective," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 2, pp. 860–874, Jun. 2022.
- [13] M. Huang, W. Liang, Z. Xu, M. Jia, and S. Guo, "Throughput maximization in software-defined networks with consolidated middleboxes," in *Proc. IEEE 41st Conf. Local Comput. Netw.*, Dubai, UAE, 2016, pp. 298–306.
- [14] M. Charikar, Y. Naamad, J. Rexford, and X. K. Zou, "Multi-commodity flow with in-network processing," in *Proc. 4th Int. Symp. Algorithmic Aspects Cloud Comput.*, Helsinki, Finland, 2019, pp. 73–101.
- [15] L. Mei, J. Gou, J. Yang, Y. Cai, and Y. Liu, "On routing optimization in networks with embedded computational services," 2022, *arXiv:2210.03338*.
- [16] W. Ma, J. Beltran, Z. Pan, D. Pan, and N. Pissinou, "SDN-based traffic aware placement of NFV middleboxes," *IEEE Trans. Netw. Service Manag.*, vol. 14, no. 3, pp. 528–542, Sep. 2017.
- [17] O. Alhussein et al., "Joint VNF placement and multicast traffic routing in 5G core networks," in *Proc. IEEE Global Commun. Conf.*, Abu Dhabi, UAE, 2018, pp. 1–6.
- [18] M. Golkarifard, C. F. Chiasserini, F. Malandrino, and A. Movaghar, "Dynamic VNF placement, resource allocation and traffic routing in 5 G," *Comput. Netw.*, vol. 188, Apr. 2021, Art. no. 107830.
- [19] O. S. Oubbati, M. Atiquzzaman, T. A. Ahanger, and A. Ibrahim, "Softwarization of UAV networks: A survey of applications and future trends," *IEEE Access*, vol. 8, pp. 98073–98125, 2020.
- [20] G. Cappello et al., "Optimizing FANET lifetime for 5G softwarized network provisioning," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4629–4649, Dec. 2022.
- [21] Y. Wang et al., "Service function chain scheduling in heterogeneous multi-UAV edge computing," *Drones*, vol. 7, Feb. 2023, Art. no. 132.
- [22] Z. Chen, N. Cheng, Z. Yin, J. He, and N. Lu, "Service-oriented topology reconfiguration of UAV networks with deep reinforcement learning," in *Proc. Int. Conf. Wireless Commun. Signal Process.*, Nanjing, China, 2022, pp. 753–758.
- [23] M. Alharthi, A.-E. M. Taha, and H. S. Hassanein, "Utilizing network function virtualization for drone-based networks," in *Proc. IEEE Global Commun. Conf.*, Taipei, Taiwan, 2020, pp. 1–5.
- [24] A. Al-Hourani, S. Kandeepan, and S. Lardner, "Optimal LAP altitude for maximum coverage," *IEEE Wireless Commun. Lett.*, vol. 3, no. 6, pp. 569–572, Dec. 2014.
- [25] J. Zhao, J. Liu, J. Jiang, and F. Gao, "Efficient deployment with geometric analysis for mmWave UAV communications," *IEEE Wireless Commun. Lett.*, vol. 9, no. 7, pp. 1115–1119, Jul. 2020.
- [26] J. Sabzehali, V. K. Shah, Q. Fan, B. Choudhury, L. Liu, and J. H. Reed, "Optimizing number, placement, and backhaul connectivity of multi-UAV networks," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 21548–21560, Nov. 2022.
- [27] K.-W. Chin, L. Wang, and S. Soh, "Joint routing and links scheduling in two-tier multi-hop RF-energy harvesting networks," *IEEE Commun. Lett.*, vol. 20, no. 9, pp. 1864–1867, Sep. 2016.
- [28] M. Y.-K. Chua, F. R. Yu, J. Li, Y. Zhou, and L. Lamont, "Medium access control for unmanned aerial vehicle (UAV) ad-hoc networks with full-duplex radios and multipacket reception capability," *IEEE Trans. Veh. Technol.*, vol. 62, no. 1, pp. 390–394, Jan. 2013.
- [29] D. Bharadia, E. McMillin, and S. Katti, "Full duplex radios," in *Proc. ACM SIGCOMM Conf.*, Hong Kong China, 2013, pp. 375–386.
- [30] K. Cao, J. Zhou, G. Xu, T. Wei, and S. Hu, "Exploring renewable-adaptive computation offloading for hierarchical QoS optimization in fog computing," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 39, no. 10, pp. 2095–2108, Oct. 2020.
- [31] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2329–2345, Apr. 2019.

- [32] T. Zhang, Y. Xu, J. Loo, D. Yang, and L. Xiao, "Joint computation and communication design for UAV-assisted mobile edge computing in IoT," *IEEE Trans. Ind. Inform.*, vol. 16, no. 8, pp. 5505–5516, Aug. 2020.
- [33] Y. Du, K. Yang, K. Wang, G. Zhang, Y. Zhao, and D. Chen, "Joint resources and workflow scheduling in UAV-enabled wirelessly-powered MEC for IoT systems," *IEEE Trans. Veh. Technol.*, vol. 68, no. 10, pp. 10187–10200, Oct. 2019.
- [34] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. London, U.K.: Pearson, 1993.
- [35] "DJI Mavic 3 Pro;" [Online]. Available: <https://www.dji.com/au/mavic-3-pro/specs>
- [36] A. M. Moore, "Innovative scenarios for modeling intra-city freight delivery," *Transp. Res. Interdiscipl. Perspectives*, vol. 3, Dec. 2019, Art. no. 100024.
- [37] Z. Niu, H. Liu, X. Lin, and J. Du, "Task scheduling with UAV-assisted dispersed computing for disaster scenario," *IEEE Syst. J.*, vol. 16, no. 4, pp. 6429–6440, Dec. 2022.
- [38] M. M. Azari, G. Geraci, A. Garcia-Rodriguez, and S. Pollin, "Cellular UAV-to-UAV communications," in *Proc. IEEE 30th Annu. Int. Symp. Pers. Indoor Mobile Radio Commun.*, Istanbul, Turkey, 2019, pp. 1–7.
- [39] G. Feng, X. Li, Z. Gao, C. Wang, H. Lv, and Q. Zhao, "Multi-path and multi-hop task offloading in mobile ad hoc networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 5347–5361, Jun. 2021.
- [40] B. Li, Z. Fei, and Y. Zhang, "UAV communications for 5G and beyond: Recent advances and future trends," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2241–2263, Jan. 2019.



Kefeng Wu received the BEng degree in telecommunications engineering with First Class Honours from the University of Wollongong, Australia and Tiangong University, China, in 2018. He is currently working toward the PhD degree with the University of Wollongong. His current research interests include focuses on topology control and network design in UAVs communications networks.



Kwan-Wu Chin received the BSc degree with First Class Honours and the PhD degree with commendation from the Curtin University, Australia, in 1997 and 2000, respectively. He is an associate professor with the University of Wollongong. He then spent a few years with Motorola as a senior research engineer. In 2004, he joined the University of Wollongong as a senior lecturer and was promoted to associate professor in 2011. His main research interests include developing resource allocation algorithms for computer networks. To date, he holds four United States (US) patents, and has published more than 200 conference and journal articles.



Sieteng Soh (Member, IEEE) received the BS degree in electrical engineering from the University of Wisconsin-Madison, in 1986, and the MS and PhD degrees in electrical engineering from Louisiana State University, Baton Rouge, USA, in 1988 and 1993, respectively. From 1993 to 2000, he was a faculty member with Tarumanagara University, Indonesia, where he was the director of the Research Institute from 1998 to 2000. He is currently an associate professor with the Department of Computing, Curtin University, Perth, Western Australia. His research interests include network reliability, and parallel and distributed processing.